LINKÖPINGS UNIVERSITET

TDP005 Projekt: Objektorienterat system

Designspecifikation

Författare

Oliver Brandett, olibr699@liu.se
Julia Mak, julma873@liu.se

INSTITUTIONEN FÖR DATAVETENSKAP

Höstterminen 2023
Version 1.1

2023-11-20

1 Revisionshistorik

Ver.	Revisionsbeskrivning	Datum
1.1	Detaljbeskrivning	2023-12-08
1.0	Första versionen skapad	2023-11-17

2 Detaljbeskrivning av Player

Klassen Player representerar den karaktären som spelaren kontrollerar under spelets gång. Playerklassen ärver ifrån GameObject klassen som är den klassen som alla objekt i spelet utgår ifrån. I Playerklassen anges det beteende som spelarens rymdskepp använder, så som hur den ska förflytta sig på spelplanen beroende på spelarens tangentbordsinmatning och vad som händer när den kolliderar med ett fiendeobjekt.

Konstruktorn för Player tar in en vector2f och sätter objektets position enligt värdet. Konstruktorn kallar även på Game_Objekts konstruktor som tar in en sträng som innehåller sökvägen till en bild, samt ger värden till *speed* och *rate*.

2.1 Player

Player klassen innehåller följande metoder och variabler:

- float speed - En variabel som representerar hastigheten för objektet.
- sf::Time rate - En variabel som representerar tiden mellan varje avfyrad projektil.
- float rotation - En variabel som lagrar värdet från funktionen *find_rotation()*.
- float spin - En variabel som representerar ett objekts rotation.
- bool update(sf::Time delta, World &world) - En funktion som uppdaterar objektets position samt ser om kollision har inträffat.

2.2 Relationer

Player interagerar med fyra olika klasser för att kunna utföra de uppgifterna som är givna till den.

2.2.1 Game_Object

Game_Object klassen är basklassen som Player och alla andra objekt på spelplanen är härledda ifrån. Syftet med Game_Objekt klassen är för att kunna skapa flera olika objekt med olika texturer och egenskaper.

Player ärver från Game_Object och har tillgång till följande metoder och variabler:

- sf::RectangleShape shape
- sf::Vector2f dir - En vector för att sätta x- och y-koordinater.
- virtual bool update(sf::Time time, World &world) = 0 - En pure virtual funktion som ärvs av härleda klasser.
- void render(sf::RenderWindow &window) - En funktion som ritar ut ett objekt på skärmen.

2.2.2 Projetile

Syftet med Projektile klassen är för att Player ska kunna skjuta ett projektil-objekt med samma position och rotation.

- const float speed - En variabel som representerar en konstant hastighet för en projektil.
- const float rot - En variabel som representerar rotationen som projektilen har.
- sf::Vector2 dir - En vector för att sätta x- och y-koordinater.
- bool update(sf::Time delta, World &world) override - En funktions som objektets position samt ser om kollision har inträffat.

2.2.3 Enemy

Enemy inkluderas i Player för att kunna kontrollera om player-objektet har kolliderat med ett enemy-objekt.

- const float speed;
- const double random;

2.2.4 World

World inkluderas i Player för att kunna kollision samt för att skjuta projektiler.

- void update(sf::Time delta)
- void render(sf::RenderWindow &to) - Funktion som renderar objekten som lagts till.
- void add(shared_ptr<Game_Object> object) - Funktion som lägger till objekt på spelplanen.
- std::vector<shared_ptr<Game_Object» collides_with(Game_Object &me) const - Funktion som kontrollerar kollision.
- bool is_dead() const; - Funktions som kontrollerar om spelaren har förlorat.
- sf::Vector2f player_pos() const - En funktion som hämtar positionen av det första objektet.

3 Detaljbeskrivning av Menu_State

Menu_State är den klassen som ser till att både huvudmenyn som dyker upp när man startar spelet och dör, samt pausmenyn opererar på korrekt sätt. Klassen kan hantera alla dessa fallen eftersom att den enda skillnaden mellan dem är att på pausmenyn så har spelaren ett alternativ att återuppta sin nuvarande session av spelet.

Menu_States konstruktor lägger till alla de menyalternativ som spelaren kan göra i en vector som sedan loopas igenom för att kunna renderas. Anledningen till att alternativen läggs i en vector är eftersom att det inte ska finnas något alternativ för att återuppta sin nuvarande session när man befinner sig i huvudmenyn. Detta gör konstruktorn genom att kolla på värdet av en av in parametrarna. Konstruktorn tar även in spelarens poäng som de fick under sin session för att kunna vissa upp det i menyn.

3.1 Menu_State

Klassen Menu_State innehåller följande metoder och variabler:

- void on_key_press - En funktion som hanterar tangentbordsinmatning.
- std::shared_ptr<State> update - En funktion som uppdaterar det valda menyalternativet.

- void render - En funktion som ritar ut menyn utifrån vad som har ändrats i update.
- struct Entry - En struct som som bygger upp menyalternativen.
- vector entries - En vector som innehåller alla Entry.
- void add - En funktion som lägger till en Entry i entries.
- size_t selected - Hååler koll på vilket menyalternativ som är valt.
- Font font - Det typsnitt som texten i menyn ska använda.
- bool enter_pressed - Kontrollerar om ett menualternativ har valts.
- Time delay - Hur lång tid det tar för färgen på texten av det valda menyalternativet att ändras.
- int score - Det poäng som spelaren nådde upp till under sin session.
- Text scoretext - Den text som visar spelarens poäng.
- Text titeltext - Den text som visar spelets titel.

3.2 State

State är basklassen som Menu_State ärver ifrån samt klassen som används i main funktionen för att kalla på Menu_State.

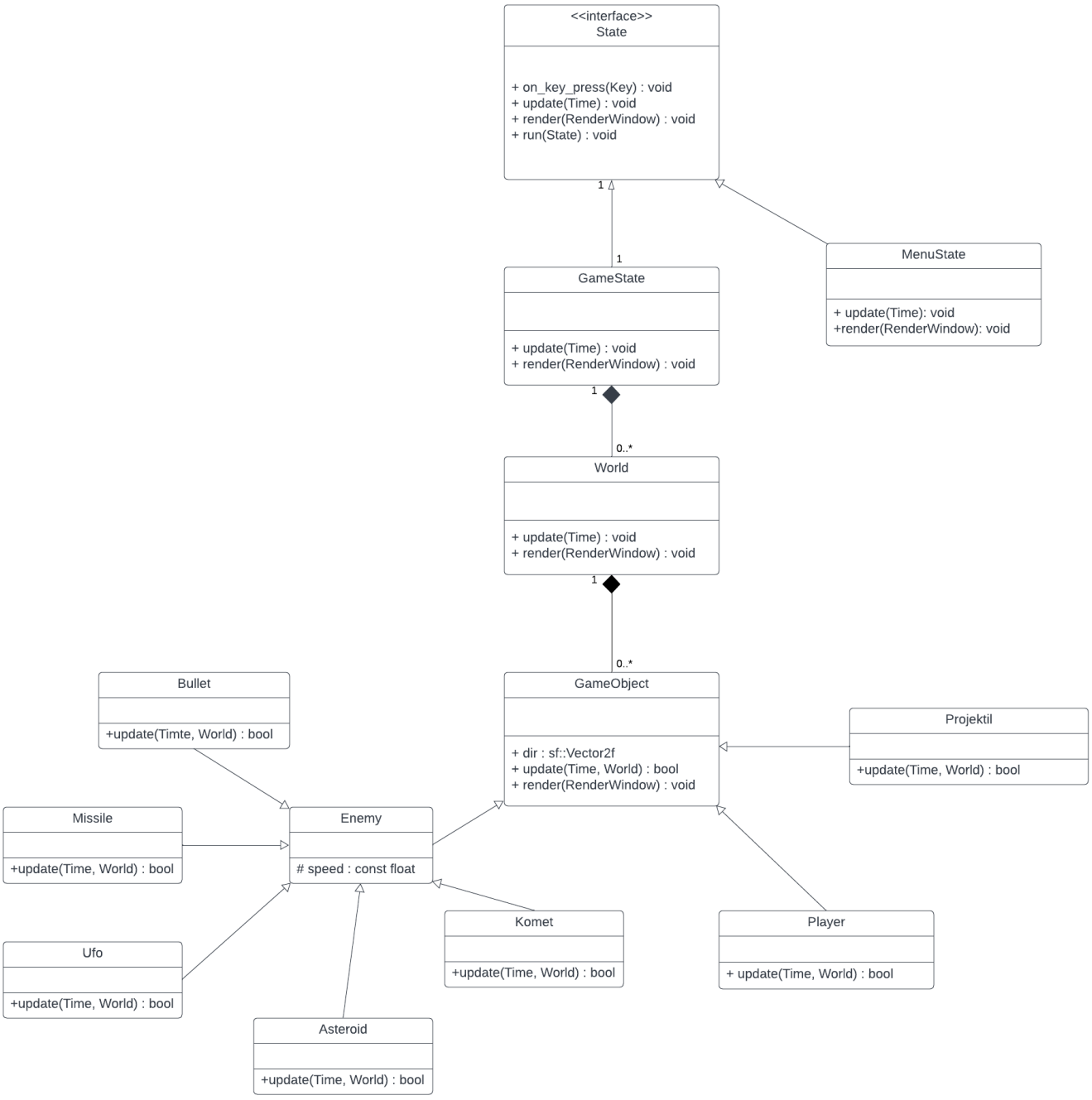
Från State ärver Menu_State:

- void on_key_press - En funktion som ärvs av härleda klasser.
- std::shared_ptr<State> update - En pure virtual funktion som ärvs av härleda klasser.
- void render - En pure virtual funktion som ärvs är härleda klasser.
- void run - En statisk funktion som kallar på de härleda klassernas update och render.

4 Beskrivning av klassdiagram

I figur 1 utgår programmet ifrån att spelet befinner sig antingen i menyn eller aktivt spelar. När spelaren står i menyn är programmet i MenuState vilket betyder att allting som sker under GameState avbryts. Detta innebär att all uppdatering av spelplanen pausas. När spelet är aktivt så sker allting på en spelplan, denna spelplan är definierad enligt World-klassen. GameObjcets är alla typer av objekt som existerar på spelplanen, anledningen till varför de alla härleder från samma basklass är för att göra uppdateringen av alla objekten så enkelt som möjligt. Ett GameObject kan inte existera utan att vara en del av en av de härleda klasserna.

Enemy-klassen är en härled klass från GameObjekt och är den klassen som alla fiendeobjekt ärver ifrån. Anledningen till att Enemy-klassen är ett mellansteg mellan GameObject och fiendeobjekten är för att göra kollisionshanteringen med Player enklare då alla fiender har samma beteende när de kolliderar med spelarens rymdskepp. Enemy-klassen gör det även enklare att i framtiden lägga till nya typer av fiender.



Figur 1: UML-diagram